# Exception Handling in the FreeST Programming Language

Bruno Faustino - 59784
Estudo Orientado
Mestrado em Engenharia Informática
Faculdade de Ciências, Universidade de Lisboa
fc59784@alunos.fc.ul.pt

## Abstract

In about 10 lines, summarise the document's key elements.

***Keywords*** exception handling, FreeST, session types, linear type systems, concurrent programming languages

## 1 Introduction

The introduction should present an overview of the project and describe the organisation of the document. The subsections presented below are suggestions for organising this chapter.

**Context.** What is the scientific/technological context of the topic of your project?

**Problem.** What is the original problem your addressed in your project?

**Motivation.** Why is this problem important and relevant? What justifies your project? How is your work on this problem different from what was already done (if something was done)?

**Goals.** What are the main goals of your project?

**Outline.** How is the rest of the document structured? The remainder of this document is organised as follows. Section 2 presents bla bla bla. ...

Exception mechanisms have been integrated in many programming languages to deal with unexpected events at runtime [4]. Unlike errors, exceptions allow a method to bring some information to the attention of the caller, propagating up the call stack until a method is found that can handle the exception [3]. Session types provide compile-time guarantees that communication between threads follows an established protocol [2]. FreeST is a concurrent programming language based on session types that is yet to properly handle failures [5].

Previous efforts have been made to handle failures in programming languages with session types [2]. This paper extends those efforts to cover all the constructs and features of the FreeST programming language.

The main goal of this project is to enhance the FreeST compiler with exceptions and exception handling mechanisms, so that FreeST programs can be robust and reliable, even in the face of failures during the communication protocols. This will also provide valuable progress to future implementations of distributed applications in FreeST, which are very prone to failures [2] (e.g. network connection problems and crashes). Additionaly, the new mechanisms must be practical from the point of view of the programmer: well integrated with the language without getting in the way, intuitive based on other programming languages, and simple without too much boilerplate code.

The remainder of this paper is organized as follows. Section 2 presents.....

## 2 Background

In this section, you should describe the scientific or technological context of your project. Provide enough information so that a reader unfamiliar with the topic can understand the problem you are addressing and the rationale for your project. This may include relevant concepts and definitions in the area, particularly those that will be used throughout this document.

To better understand the context of this project, it is important to highlight some definitions that will be used throughout this paper, namely exceptions, exception handling and the FreeST programming language.

Exceptions [4] are unexpected runtime events that lead to undesirable behaviour, which can result from code or logic errors, invalid user input, hardware or resource problems such as running out of memory, and external factors, for instance, network connection problems. Exceptions encapsulate information about the cause of the event and other information that can be useful for debugging the program (e.g. the location of the event in the code).

Exception handling [4] allows the issue to be fixed at runtime so that the program can resume execution, or terminate gracefully without causing harm. Common ways to handle exceptions [1] include logging the error message, rethrowing the exception to the caller, continuing a loop, stopping the execution of the method and returning to the caller, rollbacking the effects of the code that led to the exception, changing an object state to a static default, a combination of these, and simply ignoring the error. After executing code that can lead to an exception, it is often helpful to have code dedicated to closing resources or making asserts that the properties of the program still hold, regardless if the code caused an exception or not.

FreeST [5] is a functional programming language for safe concurrency powered by context-free session types. In FreeST, processes communicate by passing messages through bidirectional channels, which maintain a buffer of messages. A channel is composed of two endpoints, one for each side of the communication. Each channel follows a communication protocol that is itself a type that can be composed sequentially (e.g. ?Int ; !Bool is a protocol type for a session type that receives an integer and sends a boolean). The protocol described by one endpoint of a session type is the dual counterpart of the other endpoint (e.g. when one endpoint is used to send an integer - !Int - the other is used to receive an integer - ?Int). Communication is safe because these endpoints are linear, which means they must be used exactly once, and sending or receiving from an endpoint returns the endpoint that carries the continuation of the protocol (e.g. after sending an integer to an endpoint of type !Int ; ?Bool, an endpoint of type ?Bool is returned, which must, in turn, be used exactly once as well). This way, the communication on well-typed programs has no mismatched or missing messages. Output operations on channels are asynchronous, while input operations are synchronous, blocking until the buffer contains a message. At the end of communication on linear channels, one endpoint is responsible for closing the channel (output), while the other must wait for the channel to close (input). A complete protocol for a channel could then be, for example, !Int ; ?Bool ; Close and its dual type would be ?Int ; !Bool ; Wait. FreeST protocols can also contain choices which allow the sender side to select the label $\ell$ that corresponds to the path $T$ it wants to continue on (type of the sender: $\oplus\{\ell_1 : T_1,\ \ell_2 : T_2,\ \ldots\}$ , type of receiver: $\&\{\ell_1 : \text{Dual } T_1,\ \ell_2 : \text{Dual } T_2,\ \ldots\}$). FreeST includes BIFs (Built-in Functions) to manipule channel endpoints. **send** and **receive** send and receive from a linear channel. **sendType** and **receiveType** send and receive types from a linear channel. **close** and **wait** close and wait for the closing of linear channels. **select** selects the label from a set of choices from which to continue the protocol and a **case** expression can be used on the other side to handle each possible choice. Besides linear channels, FreeST also provides shared channels, which are unrestricted, so they can be used any number of times (including 0). These are often used to establish a linear session between a client and server by sending and receiving a linear channel inside a shared channel, shared by multiple clients and servers. Shared channel types start with a '$*$' symbol (e.g. $*$!Int and $*\&\{\ell : T\}$) and there is no protocol for sending / receiving types and closing / waiting. Shared channel BIFs end with a '_' symbol (**send_**, **receive_** and **select_**) and the **case** expression is also available for unrestricted choices on the receiver side. FreeST types have types themselves, called kinds. Kinds are composed of a base kind (**T** for any type, **S** for session types, and **C** for channel types, which are closed at the end) and multiplicity (lin for linear and un for unrestricted).

## 3 Related Work

This section should present the state of the art on the topic of your project. It should discuss relevant related work and existing solutions, highlighting their main contributions as well as their limitations, and identifying the gaps or opportunities that motivate your project.
Preparing this section will require you to include references to academic papers, books, and possibly online resources. The next paragraph exemplifies how to do it.

In this work, you are expected to follow the guidelines on document preparation presented in Lamports book on LaTeX [**?** ]. For editing, you may use tools such as the online platform Overleaf [**?** ]. There is also a good chance that your project will build upon some of Lamports many scientific contributions, such as the concept of logical clocks [**?** ].

## 4 ńOther Section(s) as Appropriateż

The report should include one or more sections providing a detailed description of the problem you are addressing in the project and your plan to tackle it. Use appropriate section titles for what is presented. You should explain the methods you are planning to use, or have already started to apply, in your project. This discussion should be grounded in the related work, your own understanding of the problem, and, when available, preliminary results.
In case you already have some preliminary results, consider to include a section devoted to them. This section should describe the work already carried out, what data has already been collected, what analysis and designs have already been done, what methods have been used, what programs and/or preliminary results already exist, etc.

## 5 Forthcoming Work and Conclusions

This section should include subsections describing the work to be carried out during the remainder of the school year and its objectives. It should also present a chronological plan for the completion of the project. Finally, include a concluding subsection that summarizes the contributions already made, provides a preliminary self-assessment of the progress achieved so far, and discusses the main difficulties encountered.

## References


[1] Bruno Cabral and Paulo Marques. 2007. Exception handling: a field study in Java and .NET. In *Proceedings of the 21st European Conference on Object-Oriented Programming* (Berlin, Germany) *(ECOOP'07)*. Springer-Verlag, Berlin, Heidelberg, 151–175.

[2] Simon Fowler, Sam Lindley, J. Garrett Morris, and Sára Decova. 2019. Exceptional asynchronous session types: session types without tiers. *Proc. ACM Program. Lang.* 3, POPL, Article 28 (Jan. 2019), 29 pages. doi:10.1145/3290341

[3] Barbara Liskov and John Guttag. 2000. *Program Development in Java: Abstraction, Specification, and Object-Oriented Design.* Addison-Wesley, Boston, Massachusetts, USA, Chapter Exceptions, 57–76.

[4] Susana Ortega Cisneros Pedro Mejia Alvarez, Raul E. Gonzalez Torres. 2024. *Exception Handling: Fundamentals and Programming.* Springer Nature Switzerland AG, Gewerbestrasse 11, 6330 Cham, Switzerland.

[5] FreeST Team. 2023. FreeST. https://freest-lang.github.io/. Accessed: 2026-10-28.